BRADIAN DATABASE SYSTEMS

Master Product Architecture, Software Requirements Specification, Security Architecture and Implementation Programme

Version 1.0 | 12 November 2026

BDS-MASTER-001 | Architecture and Engineering Baseline

A product of Vitalworks Limited

Contents

1. Executive product vision and boundaries
2. Logical architecture and component relationships
3. Relational data model and SQL compatibility
4. OLTP transaction engine and concurrency
5. Native OLAP processing and high-volume analytics
6. Unified HTAP semantics
7. Distributed-first horizontal scaling
8. Portable service architecture and distribution
9. Bradian Studio: browser-based database administration
10. Security architecture and non-negotiable trust model
11. Recovery, backup, replication and data lifecycle
12. Data integration, interoperability and extension model
13. Operations, telemetry and performance management
14. Non-functional requirements and preliminary service targets
15. Implementation programme and release gates
16. Formal functional requirement register
17. Comprehensive acceptance and adversarial test scenarios
18. Reference documents, standards and design influences
19. Claude engineering mandate and definition of done

Document control and source basis

This master document expands the previously supplied *DER Analytics Platform — Comprehensive Product, Storage, Compute and Distributed Analytics Architecture*, version 1.0, dated 11 November 2026. That baseline provides the original proprietary columnar engine, vectorized execution, cost-based optimization, incremental data products, distributed site analytics, durability and performance goals. The independent relational DBMS, portable database service, HTAP commit model, built-in cluster routing, browser Studio and expanded security programme are **new Bradian requirements**, not features already specified in detail by the baseline. The approved Bradian horizontal falcon/database/analytics logo is retained as the visual identity, without redesign. All implementation claims must be established by tests and evidence.

Normative language: MUST denotes a mandatory product requirement; SHOULD denotes an expected default requiring a recorded exception; MAY denotes a scoped extension. Requirements carry stable identifiers. A feature is *not complete* until its correctness, authorization, fault and operational tests pass.

1. Executive product vision and boundaries

1.1 Positioning

Bradian is a database system, not merely a reporting tool. An application must be able to create a database, define related tables, enforce constraints, insert and update rows inside transactions, recover after a crash, connect through a documented protocol, and run intensive analytical SQL without exporting data to a different product. Its brand architecture separates Bradian Core (database kernel), Transaction Engine, Analytics Engine, Hybrid Query Engine, Cluster Fabric, Stream, Connect and Bradian Studio. These are logical product subsystems, not an instruction to create isolated processes for every responsibility.

The original DER focus remains a competitive strength rather than a design constraint. Bradian must support hospital and DHIS2/DER installations through adapters, but its core metadata, SQL, permissions, storage and planning are generic. Finance, commerce, government, manufacturing and other users must be able to deploy Bradian without importing health-specific schema or runtimes.

1.2 Non-goals and realistic commitments

Bradian does not claim universal superiority to ClickHouse, Doris, PostgreSQL, SQL Server or any other system. Success must be benchmarked on disclosed hardware, datasets, concurrency, correctness, consistency and recovery behaviour. Nor must Bradian pretend that a geographically disconnected installation can simultaneously provide local unrestricted writes and global synchronous serializability. The product exposes explicit strong-cluster and autonomous-federation modes, with different conflict and availability contracts.

The platform must own its persistence format, transaction semantics, catalog, distributed coordination, query planning and execution control. It may use audited libraries for cryptography, compression, networking and memory primitives; proprietary means independently controlled database architecture, not inventing vulnerable custom cryptographic algorithms. Licensing, IP ownership, patents and third-party notices require counsel review before commercial release.

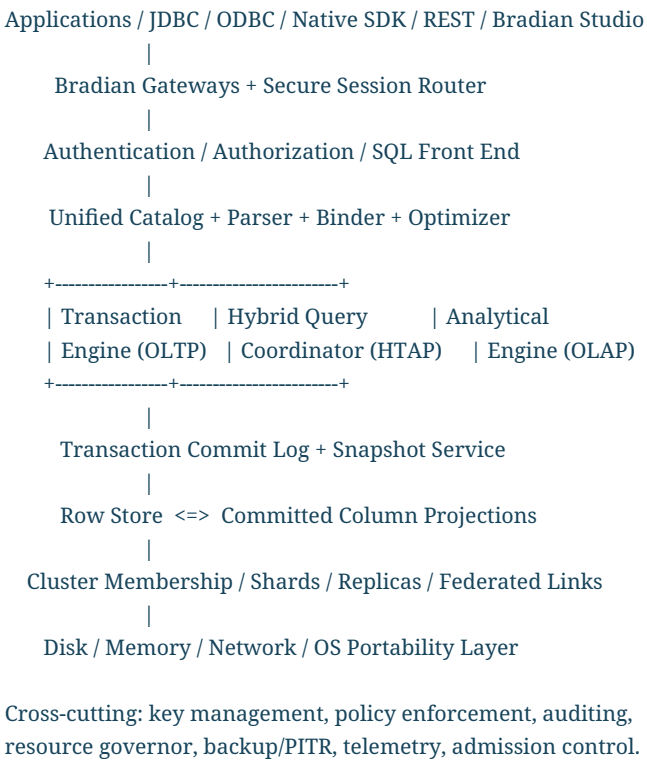
1.3 Main user groups and quality attributes

Database administrators manage creation, tuning, security, backup and recovery. Developers use SQL, drivers, migrations and observability. Data engineers ingest, transform and govern data; analysts query and publish data products; platform operators scale clusters and diagnose failures. Auditors inspect policy, access and immutable evidence. The principal quality attributes are correctness, durability,

confidentiality, availability, performance, interoperability, maintainability and portability, with security and safety overriding speed optimizations.

2. Logical architecture and component relationships

2.1 End-to-end execution structure



This architecture deliberately avoids two separately authoritative databases. The row-oriented transactional representation and committed transaction log establish authoritative state; columnar segments and materialized views are versioned derivatives. The hybrid coordinator chooses a suitable access path and must never combine incompatible snapshots. A single-node deployment may host every subsystem within one service; larger installations can assign explicit gateway, storage, transaction and analytical worker roles.

2.2 Bradian Core and platform abstraction

The core manages type definitions, value lifetimes, memory allocation, error contracts, scheduling and durable object identities. The portability layer isolates OS file APIs, directory durability, atomic rename limitations, clocks, random sources, sockets, TLS bindings, threads and system-service integration. Storage files use fixed endian and alignment rules rather than writing raw compiler structures. Bradian must build and pass filesystem crash tests independently on Windows and supported Unix-like systems.

A common logical plan and type system serves every deployment profile. CPU-specific SIMD acceleration is enabled only after detection and must have equivalent portable execution. The server must be operable with small standalone hardware as well as high-volume multi-node hardware; a mandatory Kubernetes, cloud account, object store or external load balancer would violate the standalone requirement.

2.3 Product modules and contracts

The transaction engine exposes a versioned commit and snapshot interface; the column builder consumes committed changes and publishes watermarks; the SQL front end binds names and types through the generic catalog; the optimizer requests cardinality and storage statistics; the executor consumes typed batches; and the gateway enforces authenticated session and query admission. Each cross-module contract must define wire format, errors, cancellation, version negotiation, access checks, telemetry and backward compatibility. The public interfaces must be documented separately from unstable private internals.

3. Relational data model and SQL compatibility

3.1 Databases, schemas, tables and object catalog

Bradian MUST support multiple databases and schemas, typed columns, primary and secondary indexes, ordinary views, sequences, partitions, constraints and governed ownership. The catalog maps stable object IDs to names and versions, and records dependencies before allowing a destructive schema change. The first complete SQL target should be a published, testable subset of SQL with `CREATE`, `ALTER`, `DROP`, `INSERT`, `UPDATE`, `DELETE`, `MERGE` or a clearly defined UPSERT, `SELECT`, joins, CTEs, grouping and ordering. Unsupported syntax must fail clearly and not silently discard semantics.

Identifiers require documented case folding, Unicode and quotation rules. Schema changes must preserve snapshot safety; long-running queries cannot suddenly interpret a column using an incompatible type. Online schema migration should stage new structures, backfill under controlled budgets, validate constraints and switch catalog versions atomically when possible. Dependencies among views, indexes, foreign keys and materializations must be inspectable in Studio.

3.2 Relational constraints and referential actions

Primary keys, unique keys, composite keys, `NOT NULL`, `CHECK`, defaults, identity columns and foreign keys form the integrity model. Foreign-key checks must be integrated into the same transaction as row mutations, including cascading changes, `RESTRICT` and `SET NULL` according to a precise published subset. Constraints cannot be approximated in a background analytics projection: an invalid transaction must be rejected before acknowledgement. Deferred constraint support, if offered, must be checked at commit.

`CREATE TABLE` and relationship-design screens are simply two representations of the same catalog operation. The UI must not maintain a separate metadata schema from SQL. Index-building and constraint-validation jobs report progress, old/new schema visibility and impact on concurrent write traffic. Validation failures must identify the affected constraint without exposing confidential values to unauthorized clients.

3.3 Type system and functional SQL

The mandatory core includes signed and unsigned integers with explicit overflow rules, exact decimals, IEEE floating point, booleans, UTF-8 strings, binary values, date/time/timestamp with timezone semantics, UUIDs, NULL and structured JSON. Decimal rounding, collation, Unicode comparison and timestamp normalization must be deterministic and documented. Arrays, geospatial types, vector embeddings, full-text and extension-defined types can follow as independently tested capabilities.

Bradian SQL must support parameter binding, prepared statements, scalar expressions, `CASE`, NULL semantics, subqueries, aggregate functions and window functions appropriate to the defined conformance profile. User-defined functions must execute in a restricted runtime with capability controls; SQL must not be able to invoke arbitrary OS commands or read unrelated files. SQL conformance tests and differential tests against reference systems should be part of release qualification.

3.4 Illustrative relational transaction

```
CREATE TABLE customers (
  customer_id BIGINT PRIMARY KEY,
  display_name VARCHAR(200) NOT NULL
);
CREATE TABLE invoices (
  invoice_id BIGINT PRIMARY KEY,
  customer_id BIGINT NOT NULL REFERENCES customers(customer_id),
  amount DECIMAL(18,2) NOT NULL CHECK (amount >= 0)
);
BEGIN;
INSERT INTO customers(customer_id, display_name) VALUES (1001, 'Example Customer');
INSERT INTO invoices(invoice_id, customer_id, amount) VALUES (9001, 1001, 125000.00);
COMMIT;
```

This script expresses the intended SQL surface. It is not evidence that an existing Bradian binary accepts these statements. A release gate must prove atomicity, constraint enforcement, crash durability, isolation and cross-platform recovery for these operations.

4. OLTP transaction engine and concurrency

4.1 Transaction lifecycle and ACID

The transaction manager assigns transaction identity, captures snapshot rules, tracks read/write sets, validates constraints, coordinates locks or MVCC conflicts, durably commits and publishes changes to downstream projections. BEGIN, COMMIT, ROLLBACK and savepoints must behave predictably, including failure within a multi-statement transaction. Commit acknowledgment may be issued only after the configured durability contract is met. Asynchronous replication must never be mislabeled as synchronous durability.

Atomicity means that a crash, exception or authorization failure cannot leave half an invoice or partially updated related rows visible. Consistency encompasses declarative constraints and documented invariants. Isolation must be defined by level and validated with adversarial schedules; durability depends on WAL ordering, OS flush semantics, checksums and tested recovery. Bradian must publish specific guarantees rather than rely on the acronym ACID alone.

4.2 MVCC, locking and isolation levels

A version chain or equivalent MVCC structure lets readers observe an appropriate committed snapshot while writers modify rows concurrently. Read Committed and Repeatable Read should be implemented and verified before Serializable is advertised. Serializable requires a defensible anomaly-prevention method such as serializable snapshot isolation with dangerous-structure detection or a proven locking protocol; simple snapshot isolation alone does not prevent all serialization anomalies.

The engine must provide row-level conflict management, deadlock detection or prevention, bounded lock waits, optimistic retry signals, transaction deadlines and session cancellation. Version garbage collection is constrained by active snapshots and replicas so cleanup cannot remove versions still required by readers, backups or replication. Tests must cover lost updates, write skew, phantom reads, uniqueness races, cascading deletes and concurrent DDL.

4.3 Row-oriented storage and primary indexes

The transaction store should use an independently defined portable page format, with page IDs, checksums, version metadata, free-space management and recovery-safe allocation. A B+tree or

comparable range-capable index serves primary-key and selective transactional lookups; secondary indexes are kept transactionally consistent with row mutations. Page cache, buffer eviction, hot-key handling, write batching and checkpointing are controlled by resource budgets so high ingestion does not cause unbounded memory consumption.

Row pages, index pages and WAL require careful ordering under crash and power-loss faults. Recovery must distinguish acknowledged transactions from incomplete transactions, redo committed updates where needed and undo or discard incomplete changes. A page scrubber, checksum verification and quarantined corruption mode protect against silently returning invalid data.

4.4 Connection and session semantics

Each connection has an authenticated identity, database context, transaction state, prepared statements, query limits and cancellation controls. The session service handles connection limits, idle timeouts, keepalives, credential rotation and disconnect behaviour without exposing a transaction to a different client's session. Built-in pooling may reduce per-connection cost, but must never mix security contexts or transaction-affined state. A documented binary SQL protocol and a secure HTTP administration API are separate surfaces with separate threat models.

5. Native OLAP processing and high-volume analytics

5.1 Proprietary columnar storage

The original `.derc` analytical segment concept is retained as the starting point for a versioned Bradian-native column format, with a documented migration from DER terminology. Each segment includes magic/version, schema fingerprint, column directories, typed pages, null bitmaps, dictionaries, compressed buffers, zone maps, offsets and integrity metadata. Immutable row groups enable column pruning and predictable parallel scans; atomic manifest publication makes complete segment generations visible together. Unknown critical format versions must be rejected or opened by a tested compatible reader.

Encoding should adapt to the data: dictionary encoding for repetitive strings, delta encoding for ordered integers, RLE or bit packing for small domains and vetted lossless codecs such as LZ4/Zstd for general payloads. Bradian records compression ratios and decode cost per segment, not just file size. Deleted and updated rows require version fences, delete vectors or merge-on-write semantics so old analytical copies do not reappear after compaction.

5.2 Vectorized execution and operators

The analytical runtime processes typed column batches with null masks, selection vectors and controlled ownership. Scan, filter, projection, hash aggregation, sort, top-k, distinct, join, window, time-bucket and partial/final aggregation operators execute without materializing every row into heavyweight objects. Supported SIMD paths must be benchmarked and checked against a scalar correctness oracle, including nulls, overflow, floating-point corner cases and collations.

Queries are subject to hard budgets for memory, CPU, scanned bytes, elapsed time, temporary files and output volume. Large joins and sorts spill to managed disk areas rather than terminating the server through memory exhaustion. Query cancellation must release temporary resources, and fair scheduling must keep transaction latency stable while OLAP workloads run in parallel.

5.3 Cost-based optimization and statistics

The optimizer compiles a logical tree and evaluates access paths using row counts, histograms, distinct estimates, sort order, index selectivity, cache state, compression, CPU work, disk throughput, network movement and worker availability. It performs safe predicate pushdown, partition pruning, join ordering,

materialized-view substitution and aggregation pushdown. Adaptive feedback can improve future estimates but must never change the answer, bypass authorization or produce nondeterministic schema semantics.

EXPLAIN presents the chosen plan and estimated costs, while EXPLAIN ANALYZE adds actual row counts, operator time, memory, spills, network bytes and skew. Bradian must preserve a conservative fallback plan and maintain property-based equivalence tests for optimizer transformations. Query plans and cached results must be keyed to schema, data revisions, access policy and consistency requirements.

5.4 Materialized views and streaming

The incremental engine reads committed change records, identifies impacted dependencies, retracts old contributions and applies new ones. Updating an event's date, for example, must decrease the old period aggregate and increase the new one exactly once. Distinct counts, medians, join-dependent measures and other non-invertible states require either maintained sufficient statistics or targeted recomputation. Replaying a message repeatedly must never inflate a count.

Materialized objects publish source watermark, compiler revision, data completeness, schema revision and staleness status. Streaming ingestion must support durable checkpoints, backpressure, malformed-record quarantine, replay and full rebuild. Analytical data products may provide SQL views, aggregates, cohorts, line listings and semantic APIs, but every published product has an owner, lineage, privileges, refresh contract and retirement process.

6. Unified HTAP semantics

6.1 One authoritative commit domain

The defining HTAP requirement is that every analytical representation derives from the same committed transaction history as relational tables. Bradian must maintain a durable change stream from the transaction engine to columnar projections, with monotonically ordered commit coordinates or another provably comparable revision mechanism. No reader should see an uncommitted row because a projection builder executed early. Cross-table projections and indexes require a consistent visibility barrier; a mixture of incompatible commit positions is not a valid strong snapshot.

6.2 Query consistency and projection freshness

The hybrid engine exposes explicit modes: STRONG_SNAPSHOT reads a coherent committed database snapshot; BOUNDED_STALENESS accepts a declared maximum lag; LATEST_AVAILABLE returns the newest safe analytical projection with a visible watermark. For strong queries whose projection is behind, the planner may merge committed row deltas using a correctness-proof protocol, wait within the caller deadline, or use authoritative row storage. It must not silently downgrade consistency.

For one query joining a large columnar fact table and an OLTP dimension table, both sides must resolve to a compatible snapshot and observe the same authorization policy. Projection lag and fallback cost should be observable in the query profile. Materialization errors must not make transactional rows inaccessible or destroy commit history.

6.3 Workload governance and fairness

OLTP, OLAP, ingestion, compaction, replication, backup and schema migrations compete for disk and memory. Bradian must reserve resource headroom for transactional work and enforce workload classes with budgets, priorities and preemption/cancellation policies. A dashboard refresh, large export or remote analytical scan cannot monopolize memory pools, WAL bandwidth or checkpoint I/O. Fairness and service-level objectives are monitored under mixed workload load tests rather than measured only in isolation.

7. Distributed-first horizontal scaling

7.1 Standalone, cluster and federation modes

A standalone instance creates a database in one process or service without an external coordinator. A *native cluster* presents one logical strongly governed database across storage shards and replicas connected through reliable infrastructure. *Federated installations* are independent databases that may synchronize selected information or execute privacy-governed cross-site queries. These deployment modes cannot share identical write-availability guarantees: a local installation must not lose all write capability merely because a remote national coordinator becomes unreachable.

The cluster control plane owns membership, role assignment, shard placement, replica health, compatible schema versions and administrative operations. A distributed consensus algorithm can coordinate critical membership and strongly consistent replica state. The design must specify quorum behaviour, split-brain prevention, leader terms, membership transitions and failover fencing before calling itself self-healing.

7.2 Built-in connection load balancers and query routers

Multiple healthy nodes must be able to expose database entrypoints. A client connects through a native gateway or topology-aware driver, authenticates once to the selected session, and is routed toward the correct transaction coordinator or query worker. The connection balancer considers healthy connections and saturation; the transaction router preserves shard ownership, leader epoch and session affinity; the analytical scheduler places fragments close to data; and the storage balancer moves partitions and replicas under controlled capacity policies.

Round-robin is insufficient for distributed transactions. Gateway failover must distinguish safely retryable idempotent requests from uncertain commits. Client drivers should expose a request/transaction ID and an explicit 'commit outcome unknown' error where the server cannot establish whether a commit completed. Read routing may use replicas only under the requested consistency contract and replication watermark.

7.3 Sharding and transactional replication

Shard keys and placement schemes should support automatic hash/range partitioning, online split, merge, rebalance and replica placement across independent failure domains. Writes to strongly consistent shards are replicated under a documented commit quorum and fault model. A three-voter shard commonly tolerates one voter failure while retaining a majority, but it cannot promise successful writes after loss of quorum. Data transfer during rebalancing uses checksums, snapshot-plus-catchup and fenced ownership transitions so that a shard is never served by two unsafely active leaders.

Cross-shard transactions require a transaction coordinator and a protocol for atomic commitment, crash recovery and deadlock handling. Consensus at each shard does not itself implement distributed ACID. The design must define commit records, participant recovery, coordinator loss and read visibility across shards. Distributed serializability is a separate, demanding feature gate and must not be claimed merely because transactions work on one shard.

7.4 Multi-site federation and offline reconciliation

Independent Bradian installations can authorize selected tables, streams or aggregate states for exchange. Each outbound record or delta has stable origin, object identity, revision, operation, schema version and replay idempotency. Reconnection performs dependency checks, identity matching and conflict classification; it never guesses one winner for concurrent incompatible changes. Governance policies determine whether raw data, pseudonymized data, aggregates or nothing may leave a site.

Federated analytics marks missing-site coverage and freshness; central distinct counts cannot simply sum site-local distinct people without a valid identity or mergeable state. This capability preserves DER-style offline autonomy while making the general DBMS usable for field offices, branches and regional installations.

7.5 One-command operations and safe automation

The product should provide `bradian init`, `bradian start`, `bradian cluster join`, `bradian cluster status`, `bradian cluster rebalance --plan`, `bradian cluster drain` and related commands through a versioned CLI. Node admission requires mutual authentication, a short-lived join credential, version checks and operator authorization; untrusted discovery beacons must not automatically gain membership. Scaling plans preview storage movement, expected replica health, risk and rollback limits, while production changes are audited.

Automation must avoid oscillation and destructive 'self-healing'. Before removing a replica or draining a node, Bradian verifies remaining failure-domain capacity, active snapshots and quorum. Rolling upgrades negotiate wire/storage compatibility and protect against unsafe downgrade. Scaling performance must be measured separately for throughput, query latency, write amplification and network overhead.

8. Portable service architecture and distribution

8.1 Operating-system support

Linux x86-64 is the recommended first production baseline, followed by Windows Server and Windows desktop, then macOS development/server modes and additional Unix-like systems after test evidence. The native runtime provides Windows Service, systemd and launchd integration where appropriate, while preserving the same SQL, storage format and cluster protocol contracts. Package signing, clean installation, uninstall, upgrade and data directory permissions are release-critical. Do not imply that all Unix variants are supported simply because the code is written in a portable language.

8.2 Configuration and secure defaults

A new local instance should start with a loopback-only listener, generated instance identity, no publicly accessible admin account, explicit initial administrator enrollment and protected data-directory permissions. TLS is mandatory for remote database and administrative endpoints, with mutual TLS mandatory for cluster node-to-node communication. Public exposure requires explicit bind configuration and a deployment security check. Secrets must never be written to sample configuration, command history, logs or crash diagnostics.

8.3 Packaging, upgrades and storage evolution

Binary builds must declare OS, architecture, supported filesystem, format version, protocol version and tested upgrade paths. The storage engine needs reversible metadata migrations when possible, backup requirements for irreversible format changes, safe online upgrade gates and explicit refusal of unsupported downgrade. Dependency manifests and signed release artifacts are mandatory; an installation must be able to verify update provenance before replacing an executable.

9. Bradian Studio: browser-based database administration

9.1 Product boundaries and navigation

Bradian Studio is a separate, secure web application served by or connected to Bradian, not a hidden developer console. It includes Server Overview, Database Explorer, Table Designer, Data Browser, SQL Workbench, Analytics, Data Products, Query Profiler, Cluster, Replication, Security, Backups, Audit, Monitoring, Jobs and Settings. Each route has a distinct job: lists, object details and create/edit forms

should be separate pages, while non-editing inspection can use an optional detail drawer. The global navigation must remain compact and predictable.

The visual style follows the approved Bradian identity: white and neutral surfaces, navy headings, restrained teal/blue functional accents, no gradients, readable typography, balanced tables and minimal decorative boxes. The approved horizontal logo should be used in page chrome at a modest size, with the single-colour white variant for dark backgrounds. UI layouts must work on Windows, macOS and Linux browsers and support keyboard interaction and accessible labels.

9.2 SQL Workbench and data explorer

The Workbench offers multiple query tabs, syntax highlighting, schema-aware completion, parameterized execution, cancellation, explain/profile views, plan comparison, result paging, query history and controlled exports. It must separate read-only inspection from data-changing execution with permissions and confirmation for destructive operations. Results are streamed or paged with budgets, never fully downloaded into the browser by default. Sensitive values are masked according to policy even in explain/profile and saved query history.

The Explorer navigates databases, schemas, tables, columns, constraints, foreign keys, indexes and views. Table detail shows schema, storage statistics, partitions, dependencies and permissions. Data editing occurs on a dedicated page or explicit transaction editor and honors row-level authorization, optimistic revision checks and constraint errors; an administrator UI must not bypass the database transaction engine.

9.3 Operational consoles and diagnostics

Cluster screens display actual nodes, roles, shard leaders, replica lag, quorum risk, placement, load balancing, in-flight relocations and upgrade progress. Security screens manage users, roles, grants, sessions, certificates, key rotation and audit evidence with protected workflows. Backup screens show snapshot verification and restore rehearsal results, not just filenames. Performance screens link slow queries to plan operators, storage pages, lock waits and workload admission. Every operation returns auditable action ID, outcome and recovery guidance.

9.4 Studio security baseline

Studio requires secure session cookies, CSRF defense, strong authentication/MFA for privileged users, content security policy, output encoding and strict API authorization. Browser tokens may not grant direct filesystem access, shell execution or broad cluster control. The web frontend must use parameterized query endpoints with explicit statement permissions and enforce authorization again at execution. An administrative action cannot become safe solely because its button was hidden in the UI.

10. Security architecture and non-negotiable trust model

10.1 Security objectives and threat boundaries

Security is a core database correctness property, not a feature added after SQL works. Bradian protects confidentiality of records, integrity of committed transactions and availability of authorized workloads against network attackers, compromised clients, malicious insiders, misconfigured nodes, untrusted imports, storage corruption and software supply-chain compromise. The threat model covers the public SQL listener, Studio and administrative APIs, cluster join, replication transport, local disk, backup destinations, third-party connectors, plugins, control plane and operational staff. A privileged database identity is not automatically a privileged operating-system identity.

A product security programme must maintain data-flow diagrams, trust boundaries, abuse cases, asset classification, security requirements, remediation SLAs and test evidence. Explicit trust zones include

client-to-gateway, gateway-to-coordinator, node-to-node replication, control-plane membership, backup storage, federation endpoints and build-to-release artifacts. Each boundary must authenticate the counterparty, authorize its requested action, validate untrusted input and produce meaningful audit events. Default deny is the baseline.

10.2 Identity, authentication and credential lifecycle

Bradian must integrate local identities and enterprise identity providers through supported, tested methods. Administrative login should require phishing-resistant MFA where feasible, with strong enrollment/recovery controls and time-bounded emergency access. Database drivers use secure password authentication, client certificates or approved delegated tokens; weak plaintext methods are not enabled by default. Passwords are stored only through contemporary password-hashing schemes with unique salts and tuneable work factors, never reversible encryption. Session and refresh tokens must have audience, lifetime, issuer, revocation and rotation semantics.

Provisioning and deprovisioning workflows must be auditable. A disabled identity must stop creating sessions, and sensitive long-running jobs or exports must recheck authorization before delivering results. Service accounts have explicit owners, narrow scopes and expirations; node identities are separate from human administrators. Certificate renewal and clock-skew handling must be rehearsed before deployment. Credential secrets cannot appear in server logs, query plans, process arguments or support bundles.

10.3 Authorization: RBAC, ABAC, row and column security

The authorization model should provide database, schema, table, view, column, function, sequence, job, backup, cluster-operation and audit-reader privileges. Role-based access control supplies manageable grants; attribute-based conditions allow site, organization, classification, ownership, device trust or environmental context. Row-level policies and column masking are bound into query planning and enforced at runtime, including joins, nested queries, materialized views, data exports, background jobs and cache reads.

Policies must be impossible to bypass using a lower-level analytical API, a foreign table, a restore preview or `EXPLAIN`. The system should provide explicit definer/invoker semantics for views and routines, with strict controls on privilege escalation. Every cached result is isolated by effective security context, policy revision, schema revision and snapshot. Revocation must invalidate relevant sessions, caches and pending exports according to a documented maximum enforcement delay.

10.4 Tenant isolation and multi-database separation

A deployment may contain multiple tenants, departments or customers. Bradian must define tenant identity independently of mere database naming; policy decisions and cryptographic domains must never rely on a user-controlled string alone. Default deployments isolate tenant catalogs, data directories or object namespaces, encryption keys, cache entries, temporary spills, logs and backups according to the selected isolation profile. Cluster placement and analytical exchanges must respect tenant boundaries before optimizing cost.

Resource isolation matters as much as access control: one tenant's expensive query must not starve another tenant's transactional availability. Tenant-aware quotas cover connections, CPU, memory, temporary disk, storage, network transfers, background jobs and export sizes. Security tests must attempt cross-tenant row access, cache poisoning, metadata inference, query-plan leakage, backup mix-ups and privileged operation confusion.

10.5 Encryption in transit and at rest

All remotely accessible database and Studio endpoints must use maintained TLS configurations with certificate validation, appropriate protocol versions and strong cipher suites. Cluster RPC and federation channels require mutual TLS plus explicit node authorization. Avoid silent plaintext fallback, insecure certificate skipping and unauthenticated discovery-to-join shortcuts. Session resumption, certificate rollover and key compromise response must be implemented without disrupting legitimate access unexpectedly.

At-rest protection includes database files, WAL, snapshots, manifests, temporary spill files and backups. Bradian should use envelope encryption with keys derived or unwrapped through an approved KMS/HSM or protected local keystore. Distinct key-encryption keys and data-encryption keys support controlled rotation. Encryption metadata and integrity tags must remain versioned and authenticated, and keys must never be embedded in binary artifacts or source control. Crypto primitives must come from reviewed libraries; proprietary encryption algorithms are prohibited.

10.6 Key management and secret custody

A formal key hierarchy defines root/master keys, per-instance or per-tenant key-encryption keys and rotated data keys. Secret storage uses operating-system facilities or supported external secret managers, with least-privileged retrieval and audited rotation. Recovery keys must be documented and access-controlled; losing all copies of a required key must not be presented as recoverable by routine restore. The product must support safe key rollover and re-encryption progress reporting without exposing plaintext to logs or untrusted plugins.

Key compromise playbooks require certificate revocation, session invalidation, emergency rotation, affected-backup assessment and incident reporting. A KMS outage must have a defined policy (for example, bounded cached key use for existing encrypted pages) rather than an undocumented fail-open mode. Bradian must accurately describe what encryption does and does not protect when the host kernel or a privileged process is compromised.

10.7 Secure transport, query and input surfaces

SQL parsing, binary frame decoding, HTTP JSON, CSV ingestion, native segment readers and extension loaders are untrusted-input boundaries. Parsers must enforce length bounds, recursion limits, integer overflow checks, allocation limits and strict schema validation before accepting data. Parameterized statements prevent SQL injection through application drivers; Studio must never concatenate untrusted form fields into SQL. Query text containing secrets must be redacted from telemetry, saved statements and diagnostics where possible.

SQL execution cannot invoke arbitrary operating-system commands, load uncontrolled dynamic libraries, read arbitrary host paths or open outbound sockets unless a narrowly scoped, reviewed extension explicitly receives such capability. Imported files are staged in safe directories and validated. Decompression bombs, huge schema metadata, corrupt indexes and adversarial query plans must fail in a bounded manner, preserving service availability.

10.8 Audit, evidence integrity and privacy

The audit subsystem records authentication, grants and revocations, schema mutations, SQL access to protected objects, data exports, backup/restore, key rotation, cluster membership changes and security configuration updates. Each event identifies actor, effective identity, source, request/correlation ID, target, policy decision, timestamp, node and outcome; sensitive row values are not copied indiscriminately into audit logs. Audit retention and access are governed separately from ordinary query logs.

For high-assurance environments, audit records should be integrity protected using append-only or hash-chained batches with external checkpointing or equivalent tamper evidence. This does not make a fully compromised administrator incapable of deleting every local log, so deployments should support prompt off-node export to protected audit storage. Privacy controls include retention, deletion requests, minimization, masking, lineage for derived views, backup expiry and data-residency checks. Legal compliance must be assessed per deployment and jurisdiction rather than assumed from a technical feature list.

10.9 Cluster and federation security

A new Bradian node may join only through an authenticated enrollment workflow, with certificate identity, signed or protected join token, explicit allowed role, cluster ID, protocol version and admission policy. Peer discovery does not itself grant authority. Compromised, stale or removed replicas must be fenced from accepting writes. Consensus metadata, shard transfer and replica traffic use mutual authentication and replay protection; quorum loss must never trigger an unsafe automated split-brain promotion.

Federation has an additional trust boundary: each participating installation decides which information can be exported and under what policy. Site-to-site links need a federated identity, scopes, dataset contracts, schema compatibility, encrypted transfer, idempotent replay and audit. A query optimizer cannot send protected patient-level or commercially restricted data to another jurisdiction merely because remote computation is cheaper. Remote health status and result completeness must be visible to operators and users.

10.10 Platform and host hardening

Bradian should run as a dedicated unprivileged service account, with minimal filesystem rights, protected data/config directories, restricted network listeners and optional sandboxing. Linux hardening should address systemd sandbox directives, privilege dropping and capabilities; Windows packaging should use suitable service identities and filesystem ACLs. Administrative access must be separated from database workload access. No installation should ship universal default credentials, open remote superuser access or an unauthenticated Studio endpoint.

The configuration validator should identify exposed listeners, disabled encryption, overly broad grants, weak passwords, missing backups, unsupported operating systems, unsafe plugins, expired certificates and excessive retention. Operators may inspect a machine-readable hardening report before production go-live. Bradian must document the shared-responsibility boundary: the product cannot compensate for an unpatched compromised host or insecure physical controls.

10.11 Software supply chain and secure development

The repository requires signed commits or release provenance where feasible, protected branches, independent code review for security-critical paths, dependency pinning, SBOM generation, license review and automated vulnerability scanning. CI secrets are scoped and short-lived; release signing keys are segregated from ordinary build agents. Reproducible or verifiable builds are preferred. Cryptographic code, WAL recovery, cross-shard commit, parser, authorization and key custody receive threat modeling and focused peer review.

Continuous testing includes SAST, dependency/SBOM scanning, dynamic security tests of Studio, fuzzing SQL and binary segment parsers, adversarial transaction schedules, sanitizer builds, TLS downgrade tests and isolated penetration tests. Release policies must define critical vulnerability response time, patch channels, supported versions and disclosure procedures. A high benchmark score never waives a security or data-integrity failure.

10.12 Security incident response and assurance gates

Operators need incident signals for credential abuse, excessive failed logins, privilege escalation, anomalous exports, certificate failure, audit disruption, replication tampering, unexpected code loading and suspicious network activity. The product should generate structured alerts while limiting false positives and preserving service availability. Incident workflows include isolate or drain a node, revoke credentials, retain forensic evidence, rotate keys, identify data exposure and restore from validated clean state.

The security acceptance baseline should map controls to NIST SP 800-53 Rev. 5, OWASP ASVS 5.0 for Studio/API components, and relevant CIS platform benchmarks; these are design and testing references, not certifications. Independent security assessment, cryptographic verification, documented exceptions and remediation of critical findings must precede production claims of high assurance. **No platform can responsibly promise absolute security**; Bradian must instead provide a defensible security programme with measurable controls and continual review.

11. Recovery, backup, replication and data lifecycle

11.1 Backup architecture

A transaction-consistent backup includes an authoritative data snapshot, catalog state, WAL position, identity/key references and restore manifest. Bradian should support full and incremental backups where the format permits, encryption before remote transfer, authenticated integrity metadata, retention policies and offline restore verification. Backup success means the data can be restored and queried correctly, not merely that bytes were copied. Operators should regularly run automated restore rehearsals into isolated environments.

11.2 Point-in-time recovery and disaster recovery

Point-in-time recovery replays the transaction log from a verified snapshot to a selected committed boundary while excluding uncommitted or incomplete operations. Recovery logic must resolve in-doubt distributed transactions according to its durable coordinator records; it must not arbitrarily commit or abort a transaction whose true outcome can be established. RPO and RTO targets are deployment-specific service objectives with documented measurement conditions. Regional disaster recovery must account for replica consistency, network partition and key custody.

11.3 Retention, deletion and compaction

A deletion must propagate to transaction versions, indexes, columnar projections, materializations, caches, federated replicas and eventual backup expiry, subject to legal holds and retention policies. Background vacuum, compaction and tombstone collection cannot discard data still visible to active snapshots or needed for safe recovery. Operators need a data lineage view showing where derived values exist and whether a deletion is pending, applied or blocked by a stated policy.

12. Data integration, interoperability and extension model

12.1 Client protocols and drivers

The target platform requires a versioned SQL wire protocol and first-class JDBC, ODBC and one or more language drivers. Protocol design must specify authentication negotiation, prepared statements, type encoding, streaming cursors, transaction boundaries, deadlines, cancellation and structured error codes. Proxy support and topology-aware connection routing must work without requiring Studio. Compatibility with other database protocols may be offered only after exact behaviour and limitations have been documented.

12.2 Bulk ingestion and export

Bradian must support controlled CSV, JSON and structured interchange and SHOULD add Arrow IPC and Parquet through reviewed libraries or self-contained readers. Ingestion jobs validate schema mappings, data type coercion, batch transaction scope, duplicate policies, quarantine and restart checkpoint. Large exports are asynchronous, paged or streamed, permission checked at release, rate limited and audited. External tables and connectors operate under explicit network and credential scopes.

12.3 DER, DHIS2 and specialized extensions

DER and DHIS2 support is a separately versioned adapter family rather than an implicit built-in schema. The adapter translates aggregate values, category combinations, Events, Trackers, Enrollments, Relationships, indicators, organisation hierarchies, periods and revisions into generic Bradian storage and semantic contracts. It preserves DHIS2 authority and user permissions and reports unsupported expression semantics explicitly. Other vertical integrations may follow the same plugin architecture without modifying the database kernel.

12.4 Extensibility boundaries

Extensions may add scalar functions, data connectors, storage codecs or analytical operators only through a versioned and capability-restricted API. Prefer sandboxed extension processes or memory-safe plugins for untrusted code; in-process native plugins require an explicit trust designation and version negotiation. Extensions cannot bypass authorization, impersonate the core, access arbitrary files or participate in consensus without a reviewed protocol. Failed plugins must not corrupt catalog or committed data.

13. Operations, telemetry and performance management

13.1 Observability and explainability

Bradian exposes metrics for transaction commits, aborts, lock waits, deadlocks, replication lag, quorum health, checkpoint age, WAL bytes, cache hit rate, row/column scan bytes, materialization lag, ingest queues, memory pressure and query p50/p95/p99. Traces correlate client statements with gateway routing, planner decisions and worker fragments without revealing sensitive parameter values. The query history preserves fingerprints and redacted plans under retention controls.

13.2 Workload and admission control

Administrators define workload classes for interactive transactions, BI queries, batch analytics, ingestion, compaction, backup, restore and migrations. Each class has reservations and limits for CPU, memory, concurrency, temporary storage, I/O and network. Admission control must reject or queue work with understandable diagnostics rather than permit memory collapse. Auto-tuning suggestions use measured statistics and require safe rollback; learning algorithms cannot change SQL truth or override security policy.

13.3 Service objectives and benchmark discipline

The original DER benchmark goals provide useful *aspirational baselines* for materialized query latency and ingestion freshness, but Bradian must establish new mixed-OLTP/OLAP benchmarks including transaction throughput, commit latency, distributed write contention and cross-platform crash durability. All metrics require pinned hardware, dataset, concurrency, cache state, consistency and error bars. A performance comparison is invalid if Bradian returns stale results while the reference system returns strong snapshots, or if security checks are disabled on one side.

14. Non-functional requirements and preliminary service targets

14.1 Reliability and correctness

The release criterion for acknowledged data loss under the documented durability mode is zero in the tested failure model. The engine must preserve constraints, serializability where advertised, storage checksums, recovery and analytical parity under concurrent load. Standalone production release requires crash tests at commit and checkpoint boundaries; distributed release adds leader death, stale gateway, network partitions and reconfiguration fault matrices.

14.2 Initial measurable engineering gates

Test domain	Proposed initial target or contract	Verification context
Relational correctness	100% passing SQL/transaction reference corpus	Documented SQL subset and isolation mode
Durable acknowledgement	No acknowledged transaction lost in supported crash model	Kill/restart/power-loss simulation and WAL checks
Mixed workload isolation	OLTP p95 regression below 10% at declared OLAP co-load	Same server, same data and admission policy
Interactive analytical result	p95 under 2 seconds on published 100M-fact fixture	Fixed hardware/query/concurrency/cache conditions
Incremental projection	p95 visibility under 5 seconds in local nominal profile	Commit-to-visible watermark
Scale-out throughput	Demonstrable growth with added workers before bottleneck	Publish scaling efficiency and partition layout
Security authorization	No unauthorized row, schema, export or backup disclosure in test matrix	RBAC/ABAC/tenant and revocation tests
Restore	Verified data and catalog checksum parity	Full snapshot + WAL restore rehearsal
Availability	Explicit quorum/fault-domain envelope; no blanket 100% claim	Replica loss, network split, gateway failover

These targets are engineering hypotheses rather than certified service levels. A measured deficiency is a prioritised optimization task, never permission to weaken isolation, security, constraints or correctness.

14.3 Accessibility, operability and documentation

Studio should target WCAG 2.2 AA where practicable and test keyboard, screen-reader labels, contrast, focus management, large tables and responsive layouts. CLI messages and API errors should be actionable, stable and non-sensitive. Every public setting requires documented default, risk, allowed range, reload/restart behaviour and upgrade compatibility. Administrative guides must explain not only what action to perform but what failure modes it protects against.

15. Implementation programme and release gates

15.1 Engineering sequence

Gate 0 — Repository and architecture audit. Inspect the existing `dap` repository and supplied DER analytics code/documents; map source dependencies and identify what is functional versus planned. Produce a requirements traceability matrix, threat model, architectural decision records (ADRs), executable test harness and baseline CI. Do not replace valid code merely to align naming.

Gate 1 — Portable relational kernel. Implement native server startup, data directories, generic type/catalog, portable row pages, basic B+tree and a minimal SQL `CREATE`/`INSERT`/`SELECT` slice. Exit only with persisted data, restart durability, format tests and a basic driver or CLI.

Gate 2 — Correct transactions. Add WAL, atomic commit, rollback, MVCC, locks/conflicts, constraints, crash recovery and SQL DML. Exit with concurrent anomaly tests, fault injection, recoverable interrupted transactions and audited data/permission semantics.

Gate 3 — Full core relational SQL. Implement joins, subqueries, views, schema migrations, composite constraints, prepared statements, transactional DDL policies and explain plans. Test against a published reference SQL corpus and supported isolation levels.

Gate 4 — Native columnar/OLAP engine. Preserve and port the original `.derc` column format, vector batches, compression, indexes, parallel operators, CBO, spill and incremental views. Validate against the scalar oracle and real large-data benchmarks.

Gate 5 — Unified HTAP. Establish commit-coordinate propagation, coherent cross-store snapshots, bounded staleness, strong fallback, commit-aware materialization and mixed workload reservations. Fail if an analytical plan can return mismatched revisions.

Gate 6 — Bradian Studio and external clients. Ship browser administration, secure SQL Workbench, database explorer, schema designer, query profiles, audit, backup and node diagnostics. Deliver supported JDBC/ODBC/native client contracts, not UI-only functionality.

Gate 7 — Native horizontal scaling. Implement secure membership, multiple gateways, shard routing, consensus replication, transactional failover, online rebalancing and at least a defined subset of cross-shard atomic transactions. Exit with split-brain and cluster fault evidence.

Gate 8 — Federated installations and portable packaging. Implement independently operable site databases, secure replication/federation, Windows and Unix support, installers, backup migration and compatibility/upgrade testing.

Gate 9 — Security and production qualification. Complete threat remediation, fuzzing, penetration tests, SBOM/provenance, recovery drills, rolling upgrades, data migration and benchmark publication. No release label may claim features that lack the corresponding gate evidence.

15.2 Subsystem implementation deliverables

Every gate produces working source, unit and integration tests, API/schema updates, traceability status, documentation of known limitations, reproducible benchmark artifacts, a rollback/upgrade plan and a changelog. Security-critical modules additionally require independent reviewer sign-off. The implementation agent must report blocked items truthfully and avoid describing scaffolds, unused interfaces or sample screens as production-ready.

16. Formal functional requirement register

ID	Requirement	Verification

BRD-REL-001	Native databases, schemas, relational tables and stable catalog IDs	SQL schema integration
BRD-REL-002	PK, FK, UNIQUE, CHECK, defaults and referential actions	Constraint and race corpus
BRD-REL-003	DDL, DML, prepared statements and documented SQL subset	Differential SQL tests
BRD-REL-004	Typed JSON, decimals, timezones and correct NULL semantics	Type golden corpus
BRD-TXN-001	ACID, commit acknowledgement, rollback and savepoints	Crash and transaction tests
BRD-TXN-002	MVCC snapshots and implemented isolation levels	Concurrent anomaly tests
BRD-TXN-003	Deadlock handling, lock timeouts and revision GC	Stress and long-snapshot tests
BRD-STO-001	Native row pages, primary/secondary indexes and checksums	Storage format tests
BRD-STO-002	Versioned native columnar segments and atomic manifests	Segment/read/corruption tests
BRD-STO-003	WAL, checkpoints, restore, PITR and snapshot consistency	Recovery drills
BRD-OLAP-001	Vectorized and parallel SQL operators with bounded spills	Oracle and low-memory tests
BRD-OLAP-002	CBO, statistics, EXPLAIN and plan equivalence	Rewrite property tests
BRD-OLAP-003	Incremental views with edits, deletes and replay safety	Contribution/retraction tests
BRD-HTAP-001	Coherent committed snapshots across row/column stores	Cross-store revision tests
BRD-HTAP-002	Strong, bounded-staleness and latest-safe query contracts	Lag/fallback tests
BRD-HTAP-003	OLTP resource protection during concurrent analytics	Co-load benchmark
BRD-CL-001	Secure bootstrap, join, membership and leader fencing	Admission and split-brain tests
BRD-CL-002	Native multi-gateway routing and safe retry semantics	Gateway fault tests

BRD-CL-003	Sharding, replicated write quorum and online rebalancing	Placement/failover tests
BRD-CL-004	Distributed transaction and coordinator recovery	Cross-shard crash matrix
BRD-FED-001	Independent site operation and authorized federation	WAN outage/convergence tests
BRD-FED-002	Data residency and version-aware conflict resolution	Privacy/replay tests
BRD-SEC-001	Authentication, MFA-capable privileged login and secure sessions	AuthN threat tests
BRD-SEC-002	RBAC/ABAC, row/column/tenant policies end-to-end	Unauthorized-access matrix
BRD-SEC-003	TLS/mTLS and managed at-rest encryption	Protocol/crypto tests
BRD-SEC-004	Versioned secrets, keys, rotation and revocation	Rotation/compromise drills
BRD-SEC-005	Tamper-evident audit and protected export	Integrity and audit tests
BRD-SEC-006	Fuzzed parsers, plugin isolation and secure host defaults	Fuzz/SAST/DAST suite
BRD-SEC-007	Verified incident/backup/recovery security workflows	Tabletop and restore exercise
BRD-STU-001	Browser database explorer, data editor, SQL workbench	UI E2E and auth tests
BRD-STU-002	Cluster, security, backup and performance consoles	Operational E2E tests
BRD-API-001	Versioned SQL drivers and secure administrative APIs	Contract and load tests
BRD-OPS-001	Observability, limits, cancellation and upgrade support	Reliability/load tests
BRD-PORT-001	Native Windows/Linux/macOS profiles and storage portability	Platform CI/installation tests
BRD-EXT-001	Controlled plugins and optional DER/DHIS2 adapter	Capability and compatibility tests

17. Comprehensive acceptance and adversarial test scenarios

17.1 OLTP and SQL

A payment transaction that inserts a parent, several children and a ledger adjustment must either commit all changes or leave none, even if the process crashes between WAL flush and page write. Concurrent clients inserting the same unique key cannot both succeed. A foreign-key delete concurrent with a child insert cannot leave an orphan. Long-running snapshots must remain consistent through updates and online index creation. Documented isolation levels must reject or serialize known anomalies instead of merely passing single-thread tests.

17.2 OLAP and HTAP

A high-volume analytical query must produce exactly the same supported result as the scalar reference engine under nulls, late events, deletes and repeated updates. A materialized view replayed from duplicate journal changes cannot double count. A strong query joining row and column data must use compatible commit positions even if projection building is delayed; a bounded-stale query must reveal its age. During full analytical load, priority transactional requests remain within the declared protection threshold.

17.3 Distributed reliability

A gateway may fail during a transaction, but its replacement must not silently submit a non-idempotent write twice. A shard leader lost before acknowledgement must trigger a safe outcome; a stale former leader must not accept new committed writes. Rebalancing must not lose rows, violate uniqueness or expose mixed snapshots. Loss of quorum must stop unsafe strong writes rather than split the cluster. A federation disconnected for days must preserve each site's permitted local transactions and reconcile authorized changes with explicit conflicts.

17.4 Security and privacy

A user forbidden from a tenant's table cannot retrieve its values through SQL, a cached result, a materialized view, backup preview, `EXPLAIN`, federated join or export. Revoking access during a long-running export must prevent an unauthorized final release. A fabricated node-join request, expired certificate, swapped backup, corrupted segment and malicious plugin must be rejected without privilege escalation. Failed login and unusual export activity must generate protected evidence without revealing credentials or sensitive payloads.

17.5 Portability and operations

A database created on one supported operating system must open through a tested portable format reader on another, subject to declared path/encoding constraints. Recovery must survive interrupted upgrades and disk-full checkpoint failure. Bradian Studio must not allow an unauthorized button or crafted API request to execute an administrative operation. Signed installers must be verifiable, and a fresh deployment must not expose public administration by default.

18. Reference documents, standards and design influences

The following resources inform the architecture; they are not claims of certification, nor mandatory dependencies on external database systems.

1. **Source baseline:** *DER Analytics Platform — Comprehensive Product, Storage, Compute and Distributed Analytics Architecture*, v1.0 (11 November 2026), supplied by the product owner. Original native columnar, planning, execution, incremental and distributed analytics requirements.

2. **Security controls:** NIST SP 800-53 Rev. 5, *Security and Privacy Controls for Information Systems and Organizations*: https://csrc.nist.gov/pubs/sp/800/53/r5/final
3. **Web/API security verification:** OWASP ASVS 5.0.0: https://asvs.owasp.org/
4. **Host/database hardening:** CIS Benchmarks: https://www.cisecurity.org/cis-benchmarks
5. **Secure development:** NIST Secure Software Development Framework, SP 800-218: https://csrc.nist.gov/pubs/sp/800/218/final
6. **Database concurrency design reference:** PostgreSQL MVCC documentation: https://www.postgresql.org/docs/current/mvcc.html
7. **Columnar analytical reference:** ClickHouse architecture and MergeTree documentation: https://clickhouse.com/docs/
8. **Distributed SQL architecture reference:** CockroachDB architecture documentation: https://www.cockroachlabs.com/docs/stable/architecture/overview
9. **Data interchange:** Apache Arrow specification: https://arrow.apache.org/docs/format/Columnar.html
10. **Web accessibility:** W3C WCAG 2.2: https://www.w3.org/TR/WCAG22/

19. Claude engineering mandate and definition of done

Claude or any implementation team must first read this master document and the supplied baseline, audit the existing source, inventory completed work, establish requirements-to-tests traceability, and implement one verified vertical slice at a time. Design decisions concerning file format, isolation, WAL, replication, distributed commit, encryption, plugins and SQL compatibility must be recorded as ADRs before irreversible code or persistent format decisions. Every release must expose its implemented capabilities and unsupported features accurately.

Definition of done: production-relevant features are functional in code, integrated through the native service and supported drivers, protected by authorization, covered by correctness and crash tests, documented for operators, benchmarked where performance is claimed, and operable through supported command-line and browser workflows. Incomplete stub services, a nonfunctional cluster diagram, an unaudited crypto feature or an attractive Studio screen are not deliverables by themselves.

Document status statement: This master specification establishes the intended scope and engineering acceptance conditions for Bradian Database Systems. It does not certify the security, performance, interoperability, durability or existence of any implementation. Those claims require measured release evidence.